E2E test report — HitBox Collectibles operations console

Date: 18 September 2026
Branch: `develop` @ `0378d01`
Tooling: Playwright 1.63.0, axe-core 4.13, Chrome 153 (system channel)
Target: `http://localhost:3000` (`next dev` , Next 16.3.4) against the live backend at `HITBOX_API_URL`
Suite: 214 executed, 258 written-but-not-run (see Coverage not exercised)

Result

```
214 passed      0 failed      258 skipped      98s
```

PROJECT	EXECUTED	PASSED	FAILED
`public-chromium` (desktop, no session)	194	194	0
`public-mobile` (Pixel 7, no session)	20	20	0
`authed-chromium`	0	—	—
`authed-mobile`	0	—	—

Six of the 214 passes are **expected failures** (`test.fail`) — tests that assert the *intended* behaviour, are currently red, and are recorded as defects below. They pass the run because failing is what they are declared to do. If any of them starts succeeding, Playwright reports it as an unexpected pass, which is the prompt to delete the annotation. Nothing here is skipped to make a number look better.

The whole authenticated console is untested. Everything behind `/` needs a Clerk session, none was available for this run, and the 258 specs written for it skipped with a reason rather than failing as if the app were broken. One interactive sign-in unlocks all of them — see How to run the rest.

The API layer is the exception, and it is the most valuable thing this run did prove: **all 88 endpoints the console calls were confirmed to refuse an anonymous caller**, including every endpoint that moves money, changes ownership or edits access control. That is an exhaustive check, not a sample.

What was exercised

AREA	TESTS	WHAT IT PROVES
Session wall	33	All 28 console routes bounce a signed-out visitor to `/sign-in` ; the three public routes stay public; the intended destination survives the redirect
API ingress	10	No unauthenticated call is forwarded upstream; no token, JWT or backend origin appears in any client-visible response; `__session` is not readable from page scripts

AREA	TESTS	WHAT IT PROVES
Landing page	5×2	Identity, description, the single way in, and that `<Show when="signed-in">` does not leak the console link to a visitor
Sign in / sign up	12 (+5 on mobile)	Form renders, adapts to the Clerk instance's first factors, refuses an unknown identifier in words, password reveal toggles, links between the two screens
Accessibility	11	WCAG 2.1 A/AA via axe on all three public screens, keyboard operability, focus visibility, one `h1` per page, `lang` present
Responsive	10 (×2 projects)	No horizontal overflow at 375/768/1280/1920, primary action above the fold on a phone, touch-target floor on the one input that gates the whole console
Runtime hygiene	8	No console errors, no hydration mismatch, no 4xx on first-party assets, DOM-ready inside 10s
API surface	**105**	Every one of the 88 endpoints in `src/lib/api/hooks.ts` refuses an anonymous caller; writes are stopped before any upstream call; query parameters survive the rewrite; no collection leaks a row; path traversal and unknown paths refused

Findings

Four defects plus two notes, each tied to a named test. Severity is about operator impact, not about how hard the fix is.

1. An expired session blanks the screen instead of asking for a new one — High

`src/app/api/hitbox/[...path]/route.ts:38` has an explicit branch for a missing session:

```
if (!userId) return NextResponse.json(
  { error: { code: "AUTH_UNAUTHENTICATED", ... } }, { status: 401 })
```

and `ApiError.friendly` maps that code to *"Your session has expired. Sign in again to continue."* **Neither can ever run.** `auth.protect()` in `src/proxy.ts:17` matches `/(api|trpc)(.*)` and answers first, with a 307 to `/sign-in`:

```
$ curl -i localhost:3000/api/hitbox/authz/me
HTTP/1.1 307 Temporary Redirect
location: /sign-in?redirect_url=...
x-clerk-auth-status: signed-out
```

The browser's `fetch` follows that redirect, receives the sign-in **HTML** with status **200**, so `res.ok` is true, `tryParse` returns `null`, and `request()` resolves with `null` instead of throwing. React Query caches a successful `null`.

Operator impact: Clerk session tokens live about 60 seconds and are refreshed transparently, so this is rare — but when the refresh does fail (laptop asleep, network blip, revoked session), the screen empties with no error, no prompt and no redirect. The operator sees a console that looks broken and has no way to know they simply need to sign in again.

Reproduced by: `e2e/specs/public/api-ingress.spec.ts` → "answers a JSON 401 the client can map to a message"; `e2e/specs/authed/resilience.spec.ts` → "an expired session prompts for re-auth rather than emptying the screen".

Fix direction: exclude `/api/hitbox/*` from the proxy matcher and let the route handler's own 401 stand — it already exists and is already correct. Or have the client treat an HTML content-type on an API path as an auth failure.

---

2. Muted text fails AA contrast on every raised surface — High (systemic)

```
--text-3: #6b7280   on   --surface-1: #16171c   =  3.70:1
WCAG 2.1 AA requires 4.5:1 below 18.66px. Every use is 10–13px.
```

Four nodes flagged across `/sign-in` and `/sign-up`. The count understates it badly: `--text-3` is the console's muted-text token — sidebar group labels, table meta, timestamps, field hints, empty-state copy — and nearly all of it sits on `--surface-1` panels, so **every authenticated screen inherits this**. The authenticated suite has a test that measures the true blast radius once a session exists.

The landing page reports zero contrast violations, and that is misleading: its `--text-3` copy sits over the `.ambient` gradient, so axe returns those five nodes as `incomplete` / `messageKey: "bgGradient"` — *unmeasurable*, not *passing*. Computed by hand, `--text-3` over `--ground` (`#08090a`) is ≈4.1:1, also under the floor.

Reproduced by: `e2e/specs/public/a11y.spec.ts` → "/sign-in meets AA contrast", "/sign-up meets AA contrast", "the landing page's contrast can actually be measured".

Fix direction: lift `--text-3` to roughly `#8b919c`, which clears 4.5:1 on both `--surface-1` and `--ground`. Separately, give the hero copy a solid backing so a checker can verify it at all.

---

3. Form errors are invisible to assistive technology — Medium

`Field` (`src/components/ui/primitives.tsx:332`) renders its error as a bare paragraph:

```
{error ? <p className="text-[11px] text-[var(--danger)]">{error}</p> : ...}
```

There is no `id` on that paragraph, no `aria-describedby` from the input to it, no `aria-invalid` on the input, and no live region. A screen-reader user who submits a bad address hears nothing, and on returning to the field is not told it is invalid — WCAG 2.1 **3.3.1 Error Identification** and **4.1.3 Status Messages**.

This is one component, and it backs every form in the console: sign-in, sign-up, role creation, assignments, refunds, adjustments. Fixing `Field` fixes all of them at once.

Reproduced by: `e2e/specs/public/sign-in-errors.spec.ts` → "the field error is tied to the field for assistive technology".

---

4. The same refusal is shown twice — Low

Submitting an unknown email renders *"Couldn't find your account."* twice: once in the `role="alert"` banner and once under the field.

`submitCredentials` (`src/components/auth/auth-form.tsx:164`) guards the banner with `if (!errors.fields.identifier)` — "only show a global message when there is no field-level one". The guard reads the signal in the same tick the call resolves, before `errors.fields` has been populated, so both fire. A sighted operator sees it duplicated; a screen-reader user hears it twice.

Reproduced by: `e2e/specs/public/sign-in-errors.spec.ts` → "the refusal is announced once, not twice".

---

5. Table headers are the worst instance of finding #2 — Low, but read #2 first

`DataTable` (`src/components/ui/table.tsx:56`) does the semantics right — every `<th>` carries `scope="col"`, so cells are correctly associated with their column. What it gets wrong is the colour: those headers are `text-[11px]` in `--text-3`, inside a Panel on `--surface-1`. That is finding #2 at its smallest size, on the text that tells an operator which column of a nine-column money table they are reading.

No separate fix — lifting `--text-3` resolves it. A regression guard for the semantics sits in `e2e/specs/authed/a11y.spec.ts` → "tables expose their headers to assistive technology" and will confirm it on the first authenticated run.

---

6. A harness trap worth writing down — Not an app defect

An early version of this suite set `extraHTTPHeaders: { "ngrok-skip-browser-warning" }` globally. Playwright applies that to *every* request the page makes, including the cross-origin `<script>` fetch for clerk-js. The custom header makes that request non-simple, CORS preflights it, Clerk's CDN answers the preflight with a redirect, and Chrome refuses it:

```
Access to script at 'https://…/clerk.browser.js' has been blocked by CORS policy:
Redirect is not allowed for a preflight request.
```

Clerk then never loads and the sign-in form silently does nothing — indistinguishable from a real auth outage, and it produced three phantom failures before being traced. The header is now absent, with a comment in `playwright.config.ts` explaining why it must stay absent. Noted here because the next person to debug a Clerk failure will otherwise lose the same hour.

The API surface

All 88 endpoints the console calls, transcribed from `src/lib/api/hooks.ts` into `e2e/support/api-surface.ts` and classified:

KIND	COUNT	WHAT IT MEANS	CALLED FOR REAL?
`read`	49	Safe against any environment	Yes
`write`	16	Creates or edits a record	Refusals only
`act`	23	Advances a state machine — money moves, ownership changes	Refusals only

105 tests passed against the live ingress, unauthenticated. Every endpoint — including `POST /admin/finance/payouts/:id/execute`, `POST /admin/payments/refunds/:id/process`, `DELETE /admin/authz/roles/:id` — was confirmed refused at the door, with no upstream call made and no token, JWT or backend origin in any response. Also confirmed: query parameters survive the ingress rewrite (losing them would silently return page one of everything, which looks like data rather than a bug), unknown paths and path traversal are refused, and no collection endpoint returns a row to an anonymous caller.

A further 20 authenticated contract tests are written and waiting: envelope shape, the omission rule checked at the payload level (a `null` section would collapse "not permitted" into "no data"), `pageSize` actually honoured, filters actually narrowing, masked scopes not returning unmasked emails, and no write acting on an empty body.

The workflows

Nine operator state machines, catalogued in `e2e/support/api-surface.ts` with what each one costs to get wrong:

WORKFLOW	SCREEN	STEPS	WHY IT MATTERS
Refund	`/payments/refunds`	confirm-return → approve | reject → process	`process` revokes ownership, quarantines the tag, sends money and reverses the royalty — four irreversible effects in one call
Payout run	`/finance/payouts`	schedule → approve → execute | fail	`execute` pays real money to real people
Release approval	`/release-approvals`	submit → amend → decision	An approval is a compliance record attributed to whoever made it
Order lifecycle	`/orders`	7 statuses	`PAID` and `REFUNDED` are written by the payment pipelines, never by hand
Royalty reversal	`/finance/royalties`	reverse	Cancels money someone is owed; reversing twice double-credits it
Role lifecycle	`/roles`	create → edit → deactivate → delete	Seeded roles are immutable; an assigned role cannot be deleted
Role assignment	`/team`	assign → revoke	A person holds a *list* of roles — assignment is additive
Dispute	`/payments/disputes`	open → evidence → resolve	Evidence has a deadline the platform loses the case by missing
Minting	`/products`	mint → bind-tag	Minting past supply corrupts the authenticity record

28 tests, read-only and negative — per your call. Nothing advances a workflow. What they assert is everything up to the commit point, which is where the console's own bugs live:

- the buttons **offered** equal the transitions the server **allows** (`allowedTransitions` comes back on every record), checked in both directions — an offered-but-disallowed button 403s, an allowed-but-unoffered one strands the operator;
- an illegal transition is refused by the server when asked directly — which changes nothing by definition, since the refusal *is* the assertion;
- preconditions are collected before submission: a rejection reason, an override reason, a confirmation on `process` and `execute` ;
- the two permission traps hold — `reverse` needs an exact `payment-royalty:override:global` (not `manage`), and payout actions need `manage:global` specifically;
- the ledger offers no edit or delete affordance, because ledger rows are immutable.

Success paths are written and gated behind `E2E_ALLOW_WRITES=1`, ready for a seeded environment. They will not run without it.

Coverage not exercised

258 authenticated tests are written and skipped. They are not stubs — they run against a live session and a live backend, and cover:

SPEC	TESTS	COVERS
`shell-nav.spec.ts`	9 (×2 projects)	Sidebar ↔ route agreement in both directions, active marking, back-button, mobile drawer
`routes-smoke.spec.ts`	59 (×2 projects)	Every route deep-linked cold; no unresolved skeletons; finance/payments sub-navigation
`dashboard.spec.ts`	6	The omission contract — withheld sections named as withheld, hidden figures never rendered as `0`, `null` growth as "No baseline"
`permission-gating.spec.ts`	5	UI vs `/authz/me` in both directions; the `configure`-vs-`manage` trap; the refund double-grant trap
`data-tables.spec.ts`	28	Search reaching the server, pagination, empty states, drawer focus trap, no cross-currency totals
`roles-and-team.spec.ts`	7	Seeded roles read-only, permission tree not hand-typeable, additive assignment, audit `actorRoleSnapshot`
`resilience.spec.ts`	7	500 / 403 / connection refused / 502 / HTML-as-200 / slow backend, all via request interception
`a11y.spec.ts`	21	axe across every navigable screen, plus the true reach of finding #2
`api-contract.spec.ts`	20	Envelope shape, the omission rule at the payload level, pagination, filters actually narrowing, masked scopes not leaking personal data, no write acting on an empty body
`workflows.spec.ts`	28	All nine operator state machines — read-only and negative

Also out of scope for this run, deliberately — see `TESTING-APPROACH.md`: write operations against the shared backend, account creation, the two-step media upload (no live bucket on this deployment), and CSV export (`reports-dashboards:export:global` has no consumer yet).

How to run the rest

```
npm run e2e:login
```

Opens a headed browser at `/sign-in`. Sign in once — this Clerk instance uses a one-time email code, so a human has to read the inbox; nothing in this suite types a credential or creates an account. The session is saved to `e2e/.auth/user.json` (gitignored) and reused headlessly.

```
npm run e2e:authed
```

Every assertion in the authenticated suite is written against *whatever the signed-in role actually holds* — it reads `/authz/me` and checks the UI agrees with it — so it is correct for any operator account, not just a System Admin. Running it under two different roles is worth more than running it twice under one.

SCRIPT	WHAT IT RUNS
`npm run e2e`	Public suite, desktop + mobile — needs no session, safe in CI
`npm run e2e:authed`	The 210 authenticated tests
`npm run e2e:all`	Both
`npm run e2e:login`	Interactive session capture
`npm run e2e:ui`	Playwright's watch UI
`npm run e2e:report`	Opens the last HTML report

Environment note: Playwright's bundled browser download fails behind this network (`chromium` , `chrome-headless-shell` and `ffmpeg` all errored with `code=1`). The config therefore drives the installed Chrome by default and leaves video capture opt-in. In CI, run `npx playwright install chromium` and set `E2E_CHANNEL=chromium` to pin the version.